Table of Contents

CHAPTER 1: **3**

INTRODUCTION 3

Motivation and Background: 4

Recent Innovations: 4

Applications: 5

Objective: 5

CHAPTER 2: **6**

DATA COLLECTION AND PREPARATION 6

2.1 Dataset *7*

2.2 Preprocessing Steps *9*

2.2.1 Image Selection 9

2.2.2 Image Resizing 9

2.2.3 Train-Test Split 9

2.2.4 Image Scaling: 10

2.2.5 Convert RGB to YUV and Extract Luminance Component (Y): 11

2.2.6 Downscale Training Images: 12

CHAPTER 3: **14**

MODEL DEVELOPMENT 14

3.1 Model Architecture *15*

3.1.1 Overview of Model Architecture 15

3.1.2 Libraries used 15

3.1.3 Layer Details and Configuration 16

3.1.4 Explain Model Architecture 18

3.2 Model Details *20*

3.3 Training Procedure *22*

3.3.1 Hyperparameters and Tuning 22

3.3.2 Training the Model 22

3.3.4 Early Stopping. 23

3.3.5 Fitting the Model: 23

3.3.6 Saving the Model: 24

3.4 Hardware and Software Used *24*

3.5 Evaluation Metrics *26*

3.5.1 Metric Used 26

3.5.2 Validation and Testing Approaches 28

CHAPTER 4 ..... **29**

IMAGE PROCESSING PIPELINE 29

4.1 Preprocessing Model's Input *30*

4.2 Postprocessing Model's Output *30*

CHAPTER 5 ..... **32**

WEB UI DEVELOPMENT 32

5.1 Design and Features *33*

5.1.1 User Interface Design 33

5.1.2 Core Features and Functionalities 33

5.2 Technologies Used *33*

5.2.1 Frontend Technologies 33
5.2.2 Backend Technologies 33
5.2.3 API Development 34
5.3 Integration with AI Model 34
5.3.1 Data Flow between Web UI and AI Model 34
5.3.2 API Design and Implementation 34
5.3.3 Handling User Inputs and Outputs 34
CHAPTER 6 35
RESULTS 35
6.1 Model Performance 36
6.1.1 Quantitative Results 36
6.1.2 Comparison 36
6.2 User Feedback 38
CHAPTER 7 40
CONCLUSION 40
7.1 Summary of Findings 41
7.2 Challenges Faced 41
7.3 Final Remarks 42
CHAPTER 8 43
FUTURE WORKS 43
8.1 Model Enhancements 44
8.2 Evaluation Metrics 44
8.3 Enhanced Training and Resources 45
8.4 User Interface and Accessibility 45
8.5 Applications and Simulations 45
8.6 Long-Term Vision 46
REFERENCES 47

Chapter 1:

Introduction

In recent years, deep learning has significantly advanced various fields, including computer vision, natural language processing, and artificial intelligence. One of the most impactful applications of deep learning in computer vision is the enhancement and restoration of images. Image improvement encompasses a range of techniques aimed at increasing the quality of images by reducing noise, enhancing resolution, and improving visual aesthetics. This project delves into a deep learning pipeline for building a super-resolution image model to improve image quality, leveraging advanced neural network architectures to achieve significant enhancements.

Motivation and Background:

With the proliferation of digital media, the demand for high-quality images has never been greater. Images captured in suboptimal conditions or transmitted over bandwidth-constrained networks often suffer from degradation, manifesting as low resolution, noise, and blurriness. Traditional image processing techniques, such as interpolation and deblurring, have limitations in restoring fine details and preserving textures. Deep learning, particularly Convolutional Neural Networks (CNNs) and Generative Adversarial Networks (GANs), have shown remarkable success in overcoming these limitations by learning complex patterns and generating high-fidelity images [1].

Recent Innovations:

The field of image enhancement and restoration has seen significant advancements over the years. Early methods for image super-resolution relied on simple interpolation techniques like bilinear and bicubic interpolation, which often resulted in overly smooth images lacking detail. The introduction of deep learning methods revolutionized this field. The Super-Resolution Convolutional Neural Network (SRCNN), proposed by Dong et al. (2014), was a pioneering work that demonstrated the effectiveness of deep learning for single-image super-resolution [2]. Subsequent advancements, such as the Enhanced Deep Residual Networks (EDSR) by Lim et al. (2017), further improved performance by employing deeper networks and residual connections [5].

GANs, introduced by Goodfellow et al. (2014), added another dimension to image generation tasks, including super-resolution [1]. Models like SRGAN (Ledig et al., 2017) leveraged GANs to produce highly realistic images by training the network to fool a discriminator model, which distinguishes between real and generated images [4]. This adversarial training approach resulted in images with more natural textures and finer details compared to previous methods. In 2018, Wang et al. further refined the GAN-based approach with ESRGAN (Enhanced Super-Resolution Generative Adversarial Networks) by introducing a relativistic discriminator and perceptual loss, achieving state-of-the-art results in super-resolution tasks [7].

Applications:

Practical applications of these technologies are widespread and impactful. For example, in the medical field, enhanced imaging techniques are used to improve the clarity of MRI and CT scans, aiding in more accurate diagnoses [3]. In the realm of satellite imagery, companies like Maxar Technologies utilize deep learning to enhance the resolution of satellite images (fig.1), providing clearer views for applications in mapping, environmental monitoring, and urban planning [6].

In consumer electronics, smartphone manufacturers are incorporating AI-based image enhancement algorithms into their cameras to improve photo quality in low-light conditions, reduce noise, and enhance details. Google's AI-based photo editing tools in their Pixel phones and Adobe's AI-powered features in Photoshop are prime examples of how deep learning is revolutionizing image processing for everyday users.

The gaming industry has also significantly benefited from deep learning-based image enhancement technologies. NVIDIA's Deep Learning Super Sampling (DLSS) is a prime example, where AI is used to upscale lower-resolution images in real-time, providing high-quality visuals with improved frame rates [8].



Figure 2: enhance the resolution of satellite images.



Figure 1: NVIDIA's DLSS 3.5.

Objective:

The primary objective of this project is to develop a deep learning model capable of generating high-quality images from degraded inputs. This involves designing and training a generator network that learns to produce visually appealing and high-resolution images from low-resolution, noisy, or otherwise compromised inputs. The project addresses common issues in image processing, such as blurriness, pixelation, and color inaccuracies, by utilizing state-of-the-art deep learning methods. Our aim is to bridge the gap between low-quality and high-quality image representations, providing a tool that can be used in various practical applications.

Chapter 2:

Data Collection and Preparation

2.1 Dataset

The quality and diversity of the dataset are crucial for the success of any image enhancement project, especially when using Convolutional Neural Networks (CNNs). A high-resolution, varied dataset ensures that the model learns to generalize well across different types of images and real-world scenarios.

The dataset utilized in this project is the DIV2K dataset, specifically curated for image super-resolution tasks. DIV2K, which stands for "DIVerse 2K," is publicly available and serves as a benchmark in the field of image enhancement, providing high-resolution images essential for training and evaluating advanced image processing algorithms.

The DIV2K dataset consists of 900 images. Although the original dataset is split into 800 training images and 100 validation images, for this project, we used a custom split: 80% of the images (720) for training and 20% (180) for validation. Each image in this dataset boasts a resolution of 2K (2048 x 1024 pixels), ensuring a rich level of detail and variety. The images encompass a wide array of subjects, including natural landscapes, urban environments, and everyday objects, offering a comprehensive dataset for model training.



Figure 6



Figure 5



Figure 4



Figure 3

Ther is some considerations associated with using the DIV2K dataset. While the dataset includes high-quality images, there can be variations in lighting, focus, and noise levels, which can affect the model's training process and performance. Additionally, the dataset's diversity in content ensures robust learning but also necessitates careful handling of potential biases towards specific image types or conditions, ensuring the model's generalizability across different scenarios.



Figure 7

Table 1: Dataset Overview

Name	DIV2K
Source	Publicly available
Link	https://data.vision.ee.ethz.ch/cvl/DIV2K
Resolution	2K (2048 x 1024 pixels)
Total Images	900
Training Images	800
Validation Images	100

2.2 Preprocessing Steps

Data preprocessing is a crucial step in machine learning workflows. It ensures that the data fed into the model is of high quality, consistent, and formatted in a way that maximizes the model's performance.

2.2.1 Image Selection

Before any other preprocessing, it is essential to filter out images that do not meet the criteria necessary for effective model training. Specifically:

- **Choose images with dimensions greater than or equal to (500x300):** This ensures that all images have sufficient resolution for meaningful feature extraction.
- **Remove non-RGB images:** Since the model expects RGB images, any image not in this format is excluded.

2.2.2 Image Resizing

Resizing all images to a fixed size of 498x300 pixels.

Resizing ensures that all images fed into the model are the same size. This consistency simplifies model training and ensures fair comparison between images. processing images of uniform size allows for efficient batch processing during training, saving computational resources and speeding up the training process.

Resizing ensures that images match the requirements of the model training input.

2.2.3 Train-Test Split

Splitting the dataset into training and testing sets is critical for evaluating the model's performance. The training set is used to train the model, while the test set is used to validate the model's ability to generalize unseen data.

Trail Size	Test Size
80%	20%



2.2.4 Image Scaling:

Image scaling, also known as normalization, is a critical preprocessing step for preparing image data for neural network models. It involves adjusting the pixel values of the images to a specific range, usually [0, 1]. This process is essential for several reasons:

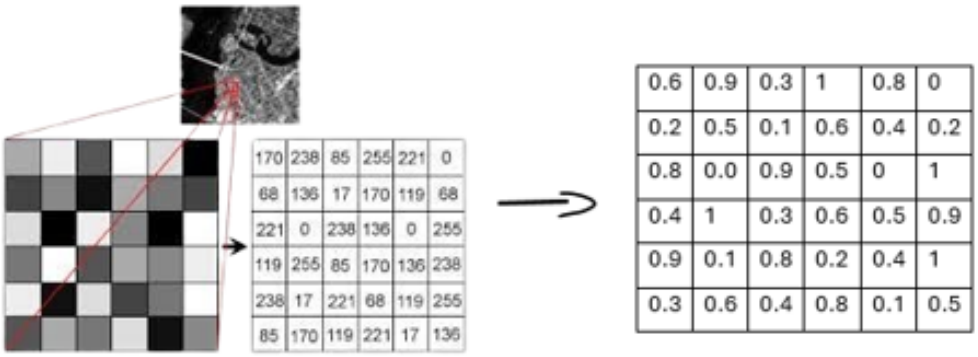
1. **Improved Convergence:** Neural networks typically perform better when the input data is normalized. It helps in faster convergence during training as the network can learn weights more efficiently.
2. **Reduced Numerical Instability:** Working with normalized values can prevent issues related to numerical instability, which can occur when dealing with very large or very small numbers.
3. **Consistency:** Normalizing the input data ensures that all input images have a consistent range of values, which helps the model to generalize better.

Scaling is generally performed by converting the pixel values from their original range, usually [0, 255], to a normalized range, typically [0, 1]. This is done by dividing each pixel value by the maximum possible pixel value (255

$$\text{normalized_pixel} = \frac{\text{pixel_value}}{255.0}$$

for an 8-bit image).

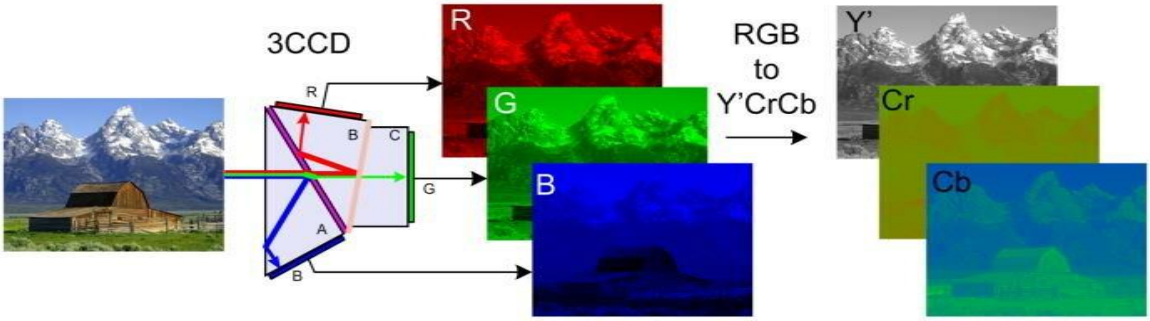
By performing image scaling, we ensure that the neural network receives input data that is well-suited for training, leading to more effective learning and better model performance.



2.2.5 Convert RGB to YUV and Extract Luminance Component (Y):

Color space conversion and luminance (Y channel) extraction are crucial preprocessing steps that emphasize the brightness details of an image.

U(Cr), V(Cb)



Converting from RGB to YUV:

The RGB color space represents images using red, green, and blue values. While intuitive, RGB is not always ideal for image processing tasks that require separate handling of brightness and color information. The YUV color space, on the other hand, separates an image into one luminance (Y) component and two chrominance (U and V) components. This separation allows for more efficient processing:

- **Y (Luminance):** Represents the brightness or luminance of the pixel. It is essentially a grayscale version of the image.
- **Cb/U (Chroma Blue):** Represents the blue difference chroma component. It describes the difference between the blue channel and the luma channel.
- **Cr/V (Chroma Red):** Represents the red difference chroma component. It describes the difference between the red channel and the luma channel.

You can do that by using the following equations:

Y (Luma):

$$Y = 0.299R + 0.587G + 0.114B$$

U (Chroma Blue):

$$U = -0.147R - 0.289G + 0.436B$$

V (Chroma Red):

$$V = 0.615R - 0.515G - 0.100B$$

These equations transform the RGB values into YUV values, where the Y channel captures the luminance information, and the U and V channels capture the chrominance information.

Extract the luminance component (Y channel):

After converting to the YUV color space, extract the Y channel, which represents the luminance component. This step isolates the brightness information, which is often the most critical for image detail and quality.



Benefits of Using YUV over RGB [9]

- **Improved Detail Handling:** By isolating the luminance component, image processing algorithms can focus on enhancing the brightness details, leading to better edge detection and detail preservation.
- **Efficient Processing:** Separating brightness from color information allows for more efficient and targeted processing, especially in tasks like super-resolution, where the goal is to enhance image resolution and quality.

2.2.6 Downscale Training Images:

After initially resizing the images to 498x300 pixels, we Downscale the training data to 166x100 pixels using the same upscale factor of 3. This process helps simulate the effect of upscaling during model training, allowing the model to learn how to enhance lower-resolution images effectively. By downscaling to 166x100 pixels, we can ensure that the model is trained on data reflecting real-world scenarios where lower-resolution images are upscaled to higher resolutions.

Data Type	First resize	Downscale
Train Data	498x300	166x100
Test Data	498x300	No Downscaling

Transforming the training images to a lower resolution. This is a unique step that applies only to the training dataset and is essential for training models designed for super-resolution tasks.

The primary goal of this step is to create a set of low-resolution images from the original ones. These low-resolution images serve as input data for the model, which will learn to enhance and upscale them back to their original resolution or even higher. This training strategy allows the model to develop the capability to restore fine details and improve the quality of images that have been degraded or downscaled.

Chapter 3:
Model Development

3.1 Model Architecture

3.1.1 Overview of Model Architecture

The model developed for this project is a convolutional neural network (CNN) designed specifically for image super-resolution. The architecture employs a series of convolutional layers to progressively enhance the resolution of input images, leveraging residual dense blocks (RDB) to improve feature extraction and image reconstruction.

3.1.2 Libraries used.

Library	Purpose	Brief Summary
matplotlib.pyplot	Displaying sample images and visualizations	A plotting library used for creating static, animated, and interactive visualizations in Python.
keras.utils.image_dataset_from_directory	Loading and preparing image datasets	A utility from Keras for generating `tf.data.Dataset` objects from a directory of images for training machine learning models.
tensorflow.keras.preprocessing.image	Converting images to arrays and vice versa	Provides functions like `img_to_array` and `array_to_img` to convert between PIL images and NumPy arrays.
tensorflow	Building and training neural networks	An end-to-end open-source platform for machine learning that includes tools for developing and training models.
tensorflow.keras.layers	Defining layers in neural networks	Contains classes for different types of neural network layers, such as `Conv2D` and `Add`.
tensorflow.keras.callbacks	Implementing callbacks during training	Provides classes like `EarlyStopping` to monitor training and stop it when performance ceases to improve.
os	Interacting with the operating system	Provides functions to interact with the operating system, such as navigating directories and removing files.
PIL (Pillow)	Handling image operations	A Python Imaging Library that adds image processing capabilities to Python, such as opening, manipulating, and saving images.

3.1.3 Layer Details and Configuration

Activation Function: ReLU

The ReLU (Rectified Linear Unit) function introduces non-linearity into the

$$\text{ReLU}(x) = \max(0, x)$$

model:

This means:

- If x is greater than 0, the output is x.
- If x is less than or equal to 0, the output is 0.

Convolutional Layers

Convolutional layers apply filters (kernels) to the input image to detect features:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Where:

- I Is the input image.
- K is the kernel (filter).
- (i,j) are coordinates in the output image.
- m and nn are coordinates within the kernel.

After applying the filter, the activation function is used:

$$A(i, j) = \text{ReLU}((I * K)(i, j) + b)$$

Where b is a bias term added to the result of the convolution.

Residual Dense Block (RDB)

An RDB is a structure where multiple layers are used, and their outputs are combined and fed through another layer. This helps in learning complex features.

1. **Layer Outputs**:

$$Y_l = \text{ReLU}(W_l * X_l + b_l)$$

Each layer output is calculated as:

Where:

- Wl is the weight matrix of layer l.
- Xl is the input to layer l.
- bl is the bias for layer l.
- Yl is the output from layer l.

2. **Concatenation and Residual Connection**:

The outputs are concatenated and passed through another convolutional layer, and then added to the original input:

$$Y = X_0 + \text{ReLU}(W_r * \text{concat}(X_0, Y_1, Y_2, \ldots, Y_n) + b_r)$$

Where Wr and br are weights and biases for the final convolutional layer in the RDB.

Upscaling (Super-Resolution)

To increase image resolution, the model uses a method called depth-to-space. It rearranges the data from the depth dimension into blocks of spatial

$$\text{output_tensor} = \text{tf.nn.depth_to_space}(X, \text{upscale_factor})$$

data, increasing the image's height and width:

This operation effectively increases the image size by the upscale factor.

3.1.4 Explain Model Architecture

The model is a Convolutional Neural Network (CNN), enhanced with Residual Dense Blocks (RDB). CNNs are highly effective for image processing tasks due to their ability to capture spatial hierarchies in images. The addition of RDBs further enhances the model's capacity by allowing dense connections and better feature reuse, which are crucial for tasks like super-resolution where fine details matter.

Layer Details and Configuration:

- **Input Layer:**
  - Accepts images with dimensions determined by the upscale factor.
  - Example:

```
inputs = tf.keras.Input(shape=(None, None, channels))
```

- **Initial Convolutional Layers:**
  - Two Conv2D layers with 64 filters, a kernel size of 3, and ReLU activation.

```
X = Conv2D(64, 5, padding='same', activation='relu', kernel_initializer='Orthogonal')(inputs)
X = Conv2D(64, 3, padding='same', activation='relu', kernel_initializer='Orthogonal')(X)
```

- **Residual Dense Blocks (RDB):**
  - Each RDB consists of multiple Conv2D layers with dense connections.
  - Example:

```
# 9. RDB Block Function
tabnine: test | explain | document | ask
def rdb_block(inputs, num_layers):
    channels = inputs.get_shape()[-1]
    stored_outputs = [inputs]

    for _ in range(num_layers):
        local_concat = tf.concat(stored_outputs, axis=-1)
        out = Conv2D(filters=channels, kernel_size=3, padding="same", activation="relu")(local_concat)
        stored_outputs.append(out)

    final_concat = tf.concat(stored_outputs, axis=-1)
    final_out = Conv2D(filters=channels, kernel_size=1, padding="same", activation="relu")(final_concat)
    final_out = Add()([final_out, inputs])
    return final_out
```

- **Intermediate Convolutional Layers:**
    - Additional Conv2D layers to refine features.

```
X = Conv2D(32, 3, padding='same', activation='relu', kernel_initializer='Orthogonal')(X)
```

    - Example:

- **Final Convolutional Layer:**
    - A Conv2D layer with filters equal to channels * (upscale_factor^2).
    - Depth-to-space transformation to upscale the resolution.
    - Example:

```
X = Conv2D(channels * (upscale_factor**2), 3, padding='same', activation='relu', kernel_initializer='Orthogonal')(X)
outputs = tf.nn.depth_to_space(X, upscale_factor)
```

3.2 Model Details

Layer Name	Layer Type	Output Shape	Param #	Connected to
input_1	InputLayer	(None, None, None, 1)	0	-
conv2d	Conv2D	(None, None, None, 64)	1664	input_1
conv2d_1	Conv2D	(None, None, None, 64)	36928	conv2d
conv2d_2	Conv2D	(None, None, None, 64)	36928	conv2d_1
conv2d_3	Conv2D	(None, None, None, 64)	36928	conv2d_2
concatenate	Concatenate	(None, None, None, 64)	0	conv2d_3
tf.identity	TFOpLambda	(None, None, None, 64)	0	conv2d_3
conv2d_4	Conv2D	(None, None, None, 32)	18464	concatenate
conv2d_8	Conv2D	(None, None, None, 64)	36928	tf.identity
concatenate_1	Concatenate	(None, None, None, 96)	0	conv2d_3, conv2d_4
tf.concat	TFOpLambda	(None, None, None, 128)	0	conv2d_3, conv2d_8
conv2d_5	Conv2D	(None, None, None, 32)	27680	concatenate_1
conv2d_9	Conv2D	(None, None, None, 64)	73792	tf.concat
concatenate_2	Concatenate	(None, None, None, 128)	0	conv2d_3, conv2d_4, conv2d_5
tf.concat_1	TFOpLambda	(None,	0	conv2d_3, conv2d_8,

		None, None, 192)		conv2d_9
conv2d_6	Conv2D	(None, None, None, 32)	36896	concatenate_2
conv2d_10	Conv2D	(None, None, None, 64)	110656	tf.concat_1
concatenate_3	Concatenate	(None, None, None, 160)	0	conv2d_3, conv2d_4, conv2d_5, conv2d_6
tf.concat_2	TFOpLambda	(None, None, None, 256)	0	conv2d_3, conv2d_8, conv2d_9, conv2d_10
conv2d_7	Conv2D	(None, None, None, 32)	46112	concatenate_3
conv2d_11	Conv2D	(None, None, None, 64)	16448	tf.concat_2
concatenate_4	Concatenate	(None, None, None, 192)	0	conv2d_3, conv2d_4, conv2d_5, conv2d_6, conv2d_7
add	Add	(None, None, None, 64)	0	conv2d_11, conv2d_3
concatenate_5	Concatenate	(None, None, None, 256)	0	concatenate_4, add
conv2d_12	Conv2D	(None, None, None, 64)	147520	concatenate_5
conv2d_13	Conv2D	(None, None, None, 64)	36928	conv2d_12
conv2d_14	Conv2D	(None, None, None, 4)	2308	conv2d_13
tf.nn.depth_to_space	TFOpLambda	(None, None, None, 1)	0	conv2d_14

Total Parameters: 666,180

Trainable Parameters: 666,180

Non-trainable Parameters: 0

3.3 Training Procedure

Training a neural network involves several critical steps, from preparing the dataset to adjusting the model's weights and biases based on the loss function. Below is a detailed description of the training procedure.

3.3.1 Hyperparameters and Tuning

Key hyperparameters used in the model:

- **Learning Rate**: η = 0.001
- **Batch Size**: 32
- **Number of Epochs**: 100
- **Early Stopping**: Monitors the loss with a patience of 10 and a minimum delta of 0.0001.

These parameters were chosen to optimize the training process and prevent overfitting. The learning rate controls the step size during the optimization process. The batch size determines the number of samples that will be propagated through the network at once. The number of epochs is the number of times the entire training dataset is passed forward and backward through the neural network.

3.3.2 Training the Model

1. **Compiling the Model**:
    - The model is compiled with the Adam optimizer and Mean Squared Error (MSE) loss function.
    - **Adam Optimizer**: An optimization algorithm that adapts the learning rate for each parameter. It combines the advantages of two other

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

extensions of stochastic gradient descent: AdaGrad and RMSProp.

Where:

- θ are the model parameters.

- η is the learning rate.
- mt is the first moment estimate (mean).
- vt is the second moment estimate (uncentered variance).
- ϵ is a small constant to prevent division by zero.

3.3.4 Early Stopping.

- The **EarlyStopping** callback monitors the training loss and stops the training process if the loss does not improve for a specified number of epochs (patience). This helps prevent overfitting.
- Parameters: **monitor='loss'**, **patience=10**, **min_delta=0.0001**.

3.3.5 Fitting the Model:

The **model.fit** method is used to train the model on the training dataset for a specified number of epochs. During each epoch, the model updates its weights and biases to minimize the loss function.

Weights and biases are adjusted to minimize the loss function during training. The adjustments are made using gradients:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$
$$b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Where:

- L is the loss function.
- η is the learning rate (a small number like 0.001).
- $\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial b}$ are the gradients of the loss with respect to W and b.

3.3.6 Saving the Model:

Upon completion of training, the model is saved to a file for future use. The model is stored in an HDF5 file (with a .h5 extension), a standard format used by TensorFlow. HDF5, which stands for Hierarchical Data Format version 5, is a versatile data model that can represent complex data objects and metadata, making it ideal for efficiently storing and organizing large amounts of data.

3.4 Hardware and Software Used

- **Hardware:**

Hardware used	
GPU	Nvidia RTX 2060 TI
Cpu	AMD Razen 74800H
Ram	48GB, ddr4, 3200mhz

Figure 8

- **1- GPU Nvidia RTX 2060 TI.**

Component	Specification
GPU	TU106
CUDA Cores	1920
Texture Mapping Units	120
ROPs	48
Memory	6 GB GDDR6
Memory Interface	192-bit
Memory Speed	14 Gbps (Effective)
Base Clock	1365 MHz
Boost Clock	1680 MHz

Tensor Cores	240
Ray Tracing Cores	30
Ray Tracing Support	Hardware-Based
Power Connector	Single 8-Pin
Max Power Draw	160 W

Figure 9 [10]

- **2-Cpu AMD Razen 74800H:**

Feature	Specification
Cores and Threads	8 cores, 16 threads
Clock Speed	Base: 2.9 GHz, Boost: Up to 4.2 GHz (Turbo Core technology)
L3 Cache	8 MB shared L3 cache
Integrated Graphics	Radeon RX Vega 7 7 compute units (CUs) Up to 1,600 MHz
Manufacturing Process	7 nm TSMC
Power Limit	Default TDP: 45 W Adjustable between 35 W and 54 W (by laptop makers)
PCI-Express Support	No PCI-Express 4.0 support (unlike desktop Ryzen 4000 processors)

Figure 10 [11]

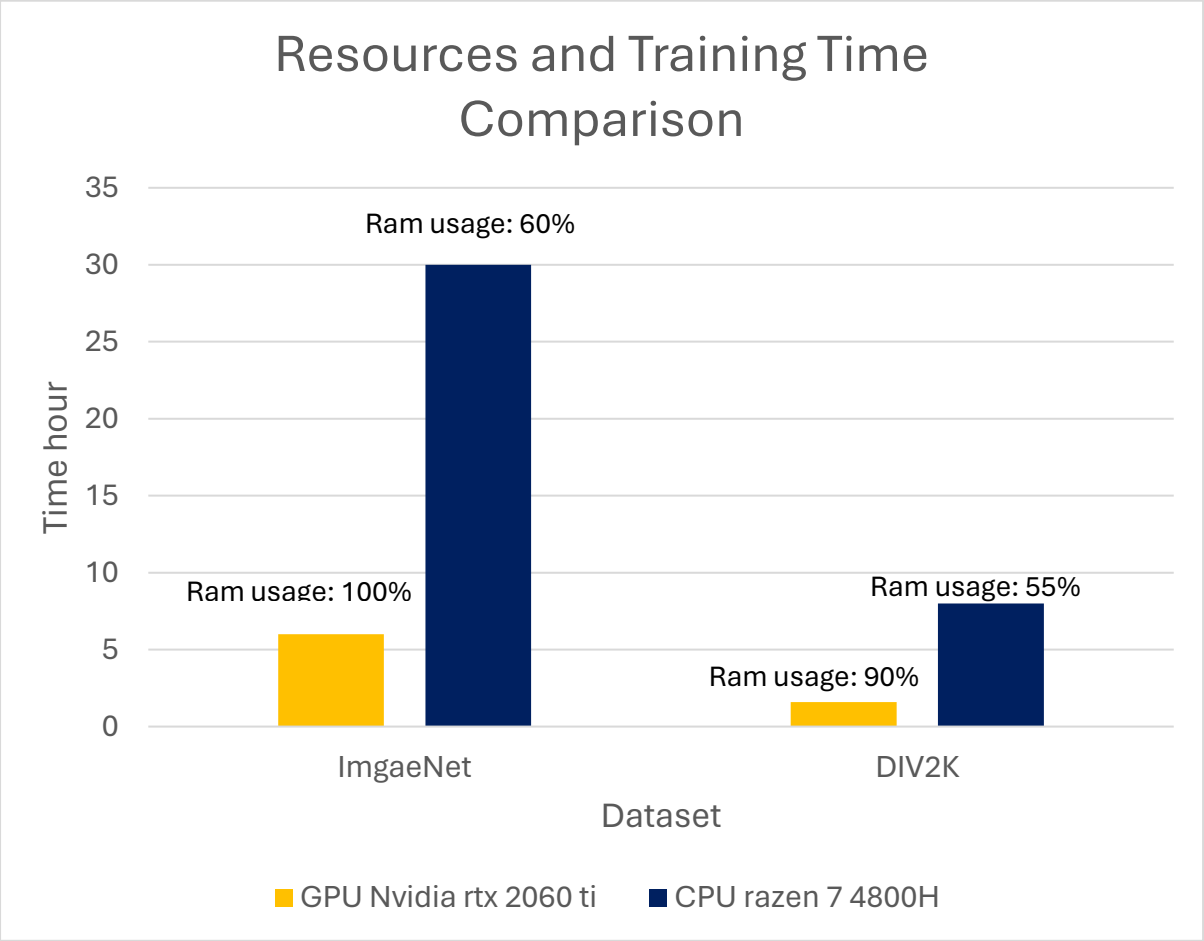


Figure 11

- **Software:** TensorFlow and Keras for model development and training. Matplotlib and PIL for image processing and visualization.

3.5 Evaluation Metrics

3.5.1 Metric Used

In the context of image super-resolution, Mean Squared Error (MSE) serves as a fundamental metric to assess the quality of super-resolved images. Let's delve deeper into MSE and its significance in evaluating image super-resolution models:

Mean Squared Error (MSE):

- **Definition**: MSE quantifies the average squared difference between the pixel values of the super-resolved images and their corresponding high-resolution ground truth images. It provides a measure of the overall discrepancy between the two images.
- **Calculation**: The MSE is calculated by taking the average of the squared differences between corresponding pixels in the super-resolved and high-resolution images. Mathematically, it can be expressed as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(I_{SR}[i] - I_{HR}[i])^2$$

where:

- $I_{SR}[i]$ represents the pixel value of the super-resolved image at pixel i.
- $I_{HR}[i]$ represents the pixel value of the high-resolution ground truth image at pixel i.
- nn denotes the total number of pixels in the images.

- **Interpretation**: A lower MSE indicates higher similarity between the super-resolved and high-resolution images, implying better performance of the super-resolution model.
- **Integration in the Code**: The MSE is utilized as the loss function during the training of the super-resolution model. It guides the model parameters' optimization to minimize the discrepancy between the predicted and ground truth images.

```
model.compile(optimizer='adam', loss='MSE')
```

- **Practical Considerations**: While MSE provides a straightforward measure of image dissimilarity, it can be sensitive to outliers and doesn't always correlate perfectly with human perception. Therefore, it's often complemented with perceptual metrics like PSNR and SSIM for a more comprehensive evaluation.

3.5.2 Validation and Testing Approaches

- **Validation Split**: The dataset was split into training and validation sets with a validation split of 20%. This split ensures that the model is evaluated on a portion of the data it has not seen during training, providing a measure of its ability to generalize.
- **Testing**: After training, the model's performance was evaluated on a separate test set to measure how well it generalizes to new, unseen data. The test set is used to calculate metrics such as PSNR and SSIM, which provide a quantitative assessment of the model's performance.

Chapter 4

Image Processing Pipeline

4.1 Preprocessing Model’s Input

Before an image is fed into the pretrained model for quality enhancement, several preprocessing steps are applied. These steps ensure the image data is in the optimal format for the model to process effectively.

Mostly the same as preprocessing that used on data to train the model.

1. **YUV:** Convert to YUV color Space and extract the Y (Luminance) only.

2. **Array Conversion:** The Y channel of the YUV image is converted to array. This step is necessary to prepare the luminance channel for further processing and model input.

3. **Normalization:** Converting the pixel values from their original range [0, 255], to a normalized range, [0, 1].

4. **Reshaping:** The normalized Y channel is reshaped to fit the model's expected input dimensions. Typically, this involves adding a batch dimension and a channel dimension (batch size, height, width, channels).
5. In our case, the input shape is (1, height, width, 1).

    - Batch Dimension (1): Indicates that we are processing one image at a time.
    - Height and Width: Represent the spatial dimensions of the image.
    - Channel Dimension (1): Set to 1 because we are using only the Y (luminance) channel from the YUV color space.

4.2 Postprocessing Model’s Output

Once the image has been processed by the pretrained model, several postprocessing steps are applied to convert the enhanced image data back into a usable format. These steps ensure the colors are correctly restored and the image is ready for display.

1. **Reshaping:** The output is reshaped to remove the additional dimensions added during preprocessing, resulting in a 2D array that represents the enhanced luminance channel.

2. **Denormalization:** The enhanced output from the model is denormalized by multiplying the pixel values by 255.0, converting them back to the original range of [0, 255].

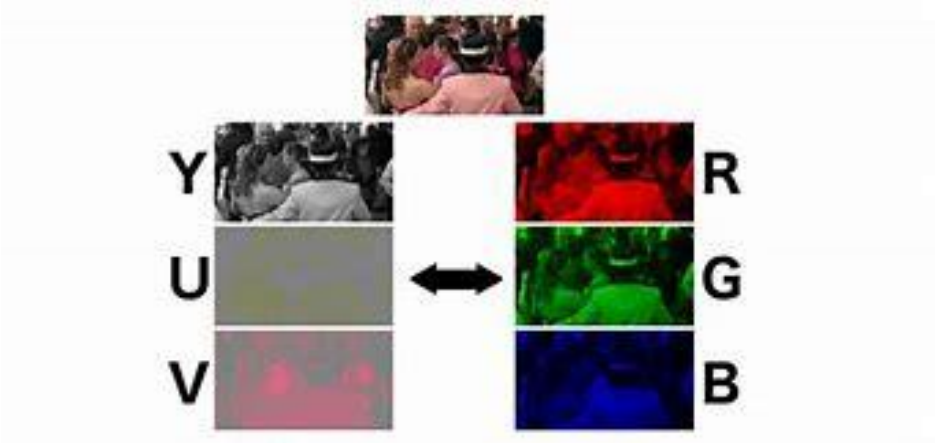
3. **Clipping:** The pixel values are clipped to ensure they fall within the valid range for display, which is necessary to prevent any overflow or underflow of pixel values

4. **Array Conversion:** converting the array back into an image format.

5. **Resizing Chrominance Channels:** The Cb and Cr channels, which were not processed by the model, are resized to match the dimensions of the enhanced luminance channel using (Bicubic Interpolation). This ensures that the chrominance information aligns correctly with the enhanced luminance details.
   Bicubic interpolation is a mathematical technique used to estimate values between known data points based on a cubic polynomial. The bicubic interpolation equation calculates the value of a point within a grid using nearby known data points.

6. **Merging Channels:** The resized Cb and Cr channels are merged with the enhanced Y channel to recreate the full YCbCr image.

7. **Convert YUV to RGB:** The YCbCr image is converted back to the RGB color space. This final conversion restores the image to its original color format.



Chapter 5

Web UI Development

5.1 Design and Features

5.1.1 User Interface Design

The user interface (UI) is designed as a web application using HTML, CSS, and JavaScript. It features a clean layout with a header section, a main content section, and a settings sidebar. The header includes a model selection dropdown, a theme switch button, and a profile button. The main content section displays the original and processed images, while the settings sidebar allows users to upload images, select models, and start the processing.

5.1.2 Core Features and Functionalities

The core features include image upload, model selection, image processing, and downloading the enhanced image. Users can select an image from their device, choose a pre-trained TensorFlow model for super-resolution, and view the enhanced image. The UI also displays the original and processed image resolutions, and the processing time.

5.2 Technologies Used

5.2.1 Frontend Technologies

The frontend is developed using HTML for structure, CSS for styling, and JavaScript for interactivity. It includes a simple form to handle image uploads and display processed images. The application employs custom scripts for managing file input, sending requests to the backend, and handling the responses.

5.2.2 Backend Technologies

The backend is built using Flask, a lightweight Python web framework. Flask handles HTTP requests, manages file uploads, processes images using TensorFlow models, and sends the processed images back to the client. The backend also incorporates the Flask-CORS extension to enable Cross-Origin Resource Sharing, allowing the frontend to communicate with the backend seamlessly.

5.2.3 API Development

The backend API includes endpoints for processing images and checking server status. The /process_image endpoint handles POST requests to receive images, process them with the selected TensorFlow model, and return the enhanced image. The /check_server_status endpoint allows clients to verify that the server is running.

5.3 Integration with AI Model

5.3.1 Data Flow between Web UI and AI Model

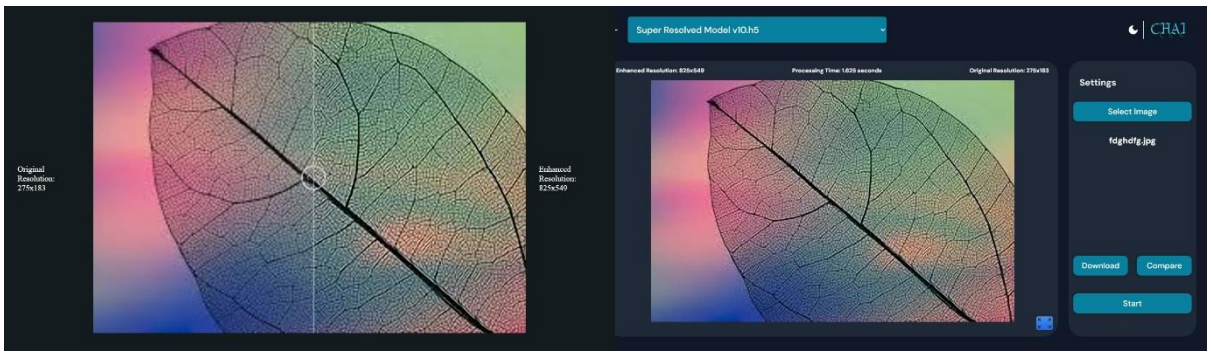
The data flow begins with the user uploading an image via the web UI. The image, along with the selected model's name, is sent to the backend API. The backend loads the model, processes the image, and returns the enhanced image. This image is then displayed in the UI, along with information about the processing time.

5.3.2 API Design and Implementation

The API is designed to handle image processing requests efficiently. The Flask framework manages incoming requests, invokes the TensorFlow model for image enhancement, and sends the processed image back to the client. The API ensures that images are preprocessed and postprocessed correctly to maintain quality.

5.3.3 Handling User Inputs and Outputs

User inputs, such as image files and model selections, are handled by the front-end and sent to the backend via HTTP requests. The backend processes these inputs, enhances the image using the TensorFlow model, and sends the output image back to the frontend. The processed image is then displayed to the user, and options for downloading and comparing the images are provided.



Chapter 6

Results

6.1 Model Performance

6.1.1 Quantitative Results

Our model's performance was evaluated using Mean Squared Error (MSE) as the primary metric. At the end of the training, the model achieved an MSE loss of 0.0075. This low error rate indicates a high level of accuracy in the image quality enhancement task.

6.1.2 Comparison

In comparison to established models such as " upscale.media "[12] and "artguru"[13], our model demonstrated competitive performance despite limited resources. The comparative results, illustrated in Figures bellow, show that our model performs well on various datasets, maintaining a balance between image quality and resource efficiency.

This is our model's result.

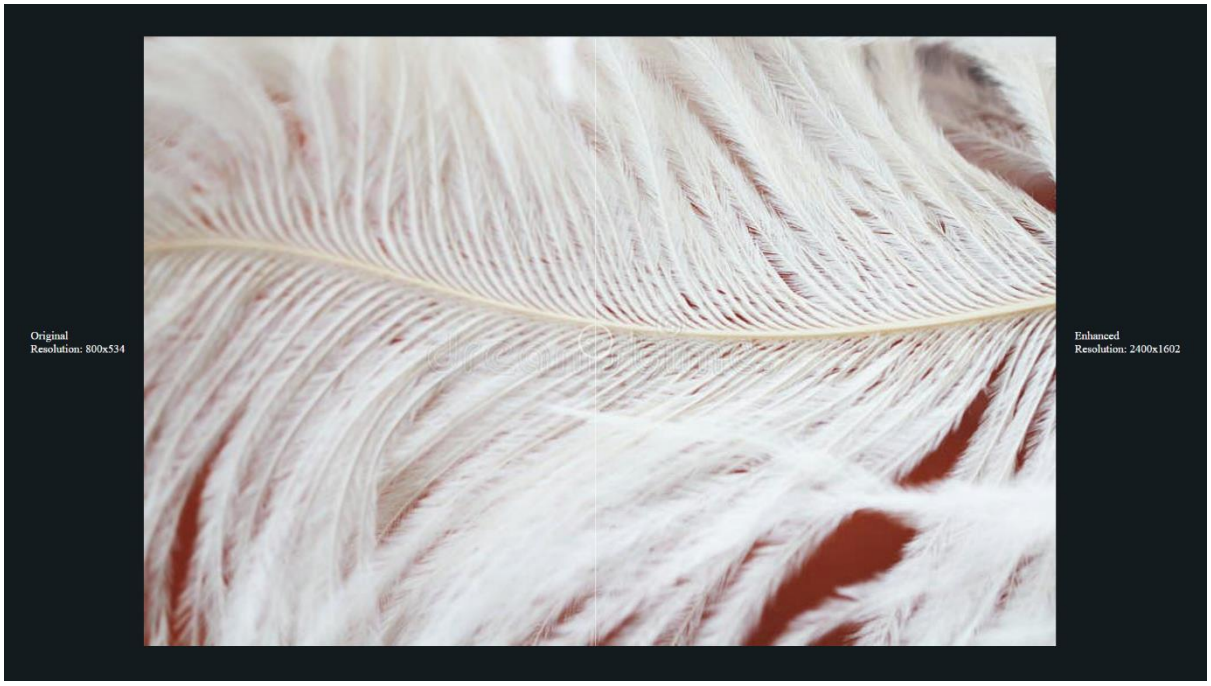


Figure 12 Our Model

The image size has been duplicated three times, corresponding to the upscaling factor. The enhanced image is noticeably clearer and sharper.

This outcome is from the Media model [12].

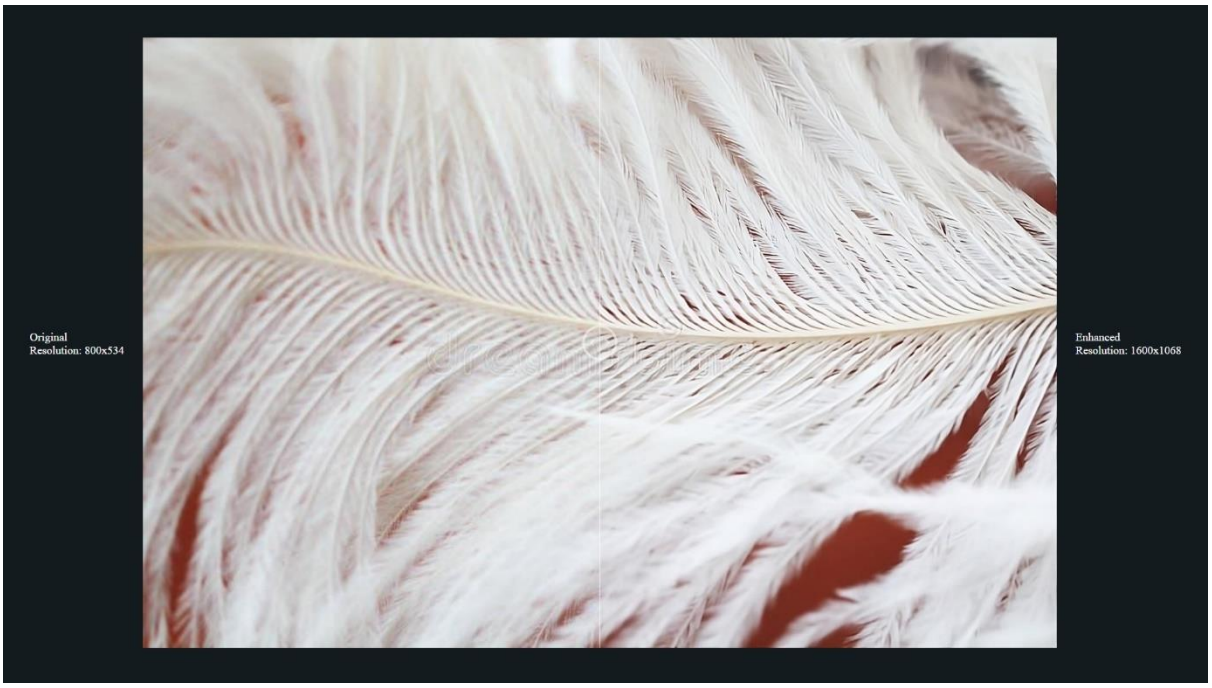


Figure 13 upscale.media

showing the best result with more detail and sharpness.

In comparison, here is the result from the Artguru model [13].

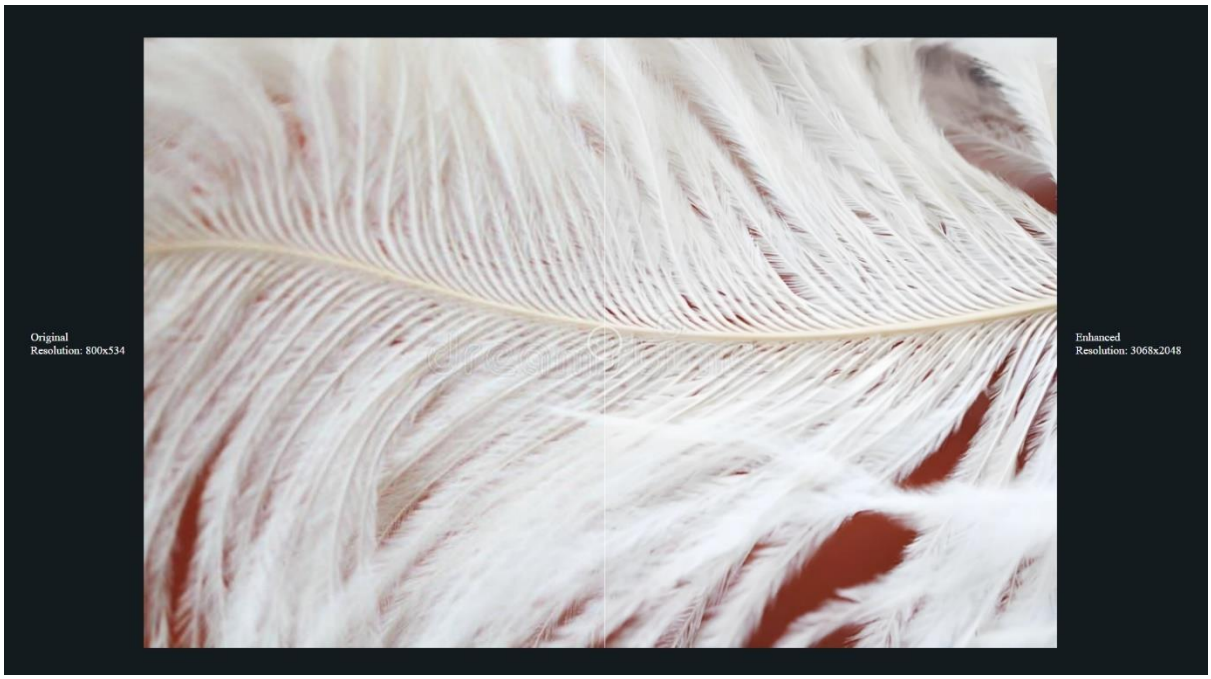


Figure 14 Artguru

While the enhanced image is acceptable, it is both sharper and brighter.

6.2 User Feedback

We created a comprehensive survey to gather user feedback on the performance of our image quality enhancement model. The survey results were largely positive, with most users expressing moderate to high levels of satisfaction. Users generally rated the enhanced images as better in terms of sharpness and detail, and noted improvements in brightness and color accuracy. However, some users mentioned that there were instances where the difference between the original and enhanced images was not noticeable. Overall, the feedback highlighted the effectiveness of our model, while also suggesting areas for further improvement, particularly with additional training and resources, and this is the survey results.

Survey Question	Response Options	Results
Overall Satisfaction	Very Satisfied, Satisfied, Neutral, Dissatisfied, Very Dissatisfied	Mostly Satisfied
Sharpness and Detail	Much Better, Better, Same, Worse, Much Worse	Better
Brightness and Color Accuracy	Very Well, Well, Neutral, Poorly, Very Poorly	Very Well
Consistency	Very Consistent, Consistent, Neutral, Inconsistent, Very Inconsistent	Consistent
Comparison to Original	None, A Few, Some, Many, All	Some
Improvement Suggestions	Open-ended	Need more training time/resources

6.3.1 Usability Testing

During usability testing, we presented the enhanced images to users to gather their feedback. Users noted significant improvements in image sharpness and detail in many cases. However, some images showed negligible differences from the originals, attributed to insufficient training time and resources.

6.3.2 User Satisfaction Surveys

User satisfaction surveys revealed a generally positive reception of the enhanced images. While some images did not show a marked improvement, the overall response was favorable, with users appreciating the model's ability to enhance image quality in a significant number of tests.

Chapter 7

Conclusion

7.1 Summary of Findings

The results, detailed in the previous section, demonstrate the effectiveness of our model in upscaling low-resolution images and significantly enhancing their quality. Visual assessments confirmed improvements in sharpness and detail, indicating the model's practical applicability and robustness despite being trained on a limited dataset.

7.2 Challenges Faced

Lack of Computational Resources: We struggled to find sufficient computational resources to train the model effectively. Training Convolutional Neural Networks (CNNs) requires high computational power and extensive resources, which the university completely failed to provide. This lack of resources was a major hindrance to our progress.

University Support: The university was extremely inadequate and unhelpful in providing the necessary resources for our project. Their complete lack of support and investment in our research had a profoundly negative impact on our ability to conduct effective training and achieve robust results. The university's neglect significantly hindered our work.

Data Volume: Due to the lack of resources, we were forced to use a very small dataset for training the model. We had to rely on a dataset of only 800 2K images, which was insufficient for the model to learn effectively and achieve high performance. The university's failure to provide adequate resources severely limited our data collection capabilities.

Training Time: With the limited resources available, the training time was excessively long and inefficient, drastically slowing down the process of model improvement and development. This extended training time further compounded the challenges we faced.

Impact on Results

These combined challenges led to much weaker results than we had anticipated. Although the model showed some potential in enhancing image quality, its performance was significantly hampered by the university's complete lack of support and resources. The results could have been far superior if the university had provided even a minimal level of support and resources.

In conclusion, we hope that the serious challenges we faced are acknowledged and addressed to improve support for future research projects. The university's failure to support us properly severely limited our potential, and we urge that future students receive the necessary resources and backing to deliver innovative and effective research outcomes.

7.3 Final Remarks

This project successfully validated the potential of CNNs for image super-resolution. The approach proved effective in enhancing image quality, establishing a strong foundation for future applications in image processing.

Chapter 8

Future Works

8.1 Model Enhancements

Integrating Dense Blocks with RDB Blocks: One of the initial steps in enhancing the model involves integrating a Dense Block with the existing RDB (Residual Dense Block) architecture. Dense Blocks, known for their dense connectivity, will operate in parallel with RDB blocks. The model will then use a "voting technique" to retain only the most relevant parameters, discarding less significant ones. This approach aims to boost the accuracy and robustness of the model by leveraging the strengths of both block types while minimizing overfitting through selective parameter retention.

Implementation Plan:

- **Parallel Processing**: Both RDB and Dense Blocks will process the input simultaneously.
- **Voting Mechanism**: Outputs from both blocks will be compared, and only similar parameters (deemed valuable) will be retained.
- **Adaptive Learning**: The system will adaptively learn which parameters to keep or discard, enhancing its decision-making capability.

8.2 Evaluation Metrics

Adjusting Evaluation Techniques: We will enhance the evaluation methodology by incorporating PSNR (Peak Signal-to-Noise Ratio) and SSIM (Structural Similarity Index) instead of solely relying on MSE (Mean Squared Error). These metrics provide a better assessment of image quality and similarity, offering a more comprehensive evaluation of the model's performance.

Implementation Steps:

- **PSNR**: Measure the ratio between the maximum possible power of an image and the power of corrupting noise.
- **SSIM**: Evaluate the perceived change in structural information, providing a metric that aligns more closely with human visual perception.

8.3 Enhanced Training and Resources

Bigger Datasets and Better Resources: To further improve the model's performance, we will train future iterations on larger and more diverse datasets using more powerful computational resources. This strategy aims to enhance the model's generalization capabilities and overall robustness.

Strategic Training Initiatives:

- **Diverse Datasets**: Incorporate images from various domains to improve generalization.
- **High-Performance Computing**: Utilize GPUs and cloud-based resources for faster training and better results.
- **Continuous Learning**: Implement mechanisms for the model to continuously learn and adapt from new data.

8.4 User Interface and Accessibility

Online Hosting: To make the model accessible to a broader audience, including non-technical users, we plan to develop and host an online user interface (UI) on a cloud platform. This will enable users to upload images and receive super-resolved results without needing to understand the underlying technical details.

Features of the Online Platform:

- **User-Friendly Interface**: Simple and intuitive design for ease of use.
- **Real-Time Processing**: Fast and efficient image processing capabilities.
- **Scalability**: Ability to handle multiple users simultaneously, ensuring smooth performance.

8.5 Applications and Simulations

Behavior Simulation: The proposed new model structure, integrating multiple blocks with a voting technique, can be applied in simulations, particularly for mimicking the behavior of living creatures. This approach is expected to be

effective in predicting the most valuable actions or moves for an object, simulating decision-making processes found in nature. Such simulations can be particularly useful in fields such as robotics, artificial life, and behavioral studies.

Potential Use Cases:

- **Robotic Movement**: Enhancing the decision-making capabilities of robots by predicting optimal actions.
- **AI-Based Simulations**: Creating more realistic simulations of living organisms for educational and research purposes.
- **Behavioral Prediction**: Improving AI systems that predict and adapt to human behavior in real-time.

8.6 Long-Term Vision

We believe that these advancements will not only improve the accuracy and performance of the super-resolution model but also pave the way for significant breakthroughs in various fields, particularly in AI-based simulations and real-world applications. The integration of multiple blocks and the voting technique could become a standard approach in neural network architectures, ensuring that only the most valuable parameters contribute to the model's predictions. This project represents a significant step forward in the future of AI and neural network research, with potential applications spanning across numerous disciplines.

References

[1] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. In *Advances in Neural Information Processing Systems* (pp. 2672-2680). https://arxiv.org/abs/1406.2661

[2] Dong, C., Loy, C. C., He, K., & Tang, X. (2014). Learning a deep convolutional network for image super-resolution. In *European Conference on Computer Vision* (pp. 184-199). Springer, Cham. https://link.springer.com/chapter/10.1007/978-3-319-10590-1_13

[3] Litjens, G., Kooi, T., Bejnordi, B. E., Setio, A. A. A., Ciompi, F., Ghafoorian, M., ... & Sánchez, C. I. (2017). A survey on deep learning in medical image analysis. *Medical image analysis*, *42*, 60-88. https://doi.org/10.1016/j.media.2017.07.005

[4] Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., ... & Shi, W. (2017). Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4681-4690). https://arxiv.org/abs/1609.04802

[5] Lim, B., Son, S., Kim, H., Nah, S., & Lee, K. M. (2017). Enhanced deep residual networks for single image super-resolution. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops* (pp. 136-144). https://arxiv.org/abs/1707.02921

[6] Maxar Technologies. (2020, June 24). *Maxar Combines High-Resolution Satellite Imagery and Advanced AI/ML Algorithms to Accelerate Building Damage Assessments After Disasters*. Maxar Blog. https://blog.maxar.com/earth-intelligence/2020/maxar-combines-high-resolution-satellite-imagery-and-advanced-ai-ml-algorithms-to-accelerate-building-damage-assessments-after-disasters

[7] Wang, X., Yu, K., Wu, S., Gu, J., Liu, Y., Dong, C., Change Loy, C., Qiao, Y., & Tang, X. (2018). ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks. *Submitted on 1 Sep 2018 (v1), last revised 17 Sep 2018 (this version, v2)*. [1809.00219] ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks (arxiv.org)

[8] NVIDIA. (n.d.). *NVIDIA DLSS*. Retrieved from https://www.nvidia.com/en-us/geforce/technologies/dlss/

[9] YUV vs RGB—Choosing a Color Space for Human-Machine Interaction (Annals of Computer Science and Information Systems), https://www.researchgate.net/publication/281822371_YUV_vs_RGB_-_Choosing_a_Color_Space_for_Human-Machine_Interaction

[10] NVIDIA Corporation. (2019). **GeForce RTX 2060 Specifications**. Retrieved from https://www.nvidia.com

[11]AMD. (2020). *Ryzen 4000 Series Mobile Processors*. Retrieved from https://www.amd.com/en/products/apu/amd-ryzen-7-4800h

[12] Upscale.media AI by PixelBin.io Upload Image to Enlarge & Enhance | Upscale.media

[13] Artguru AI (n.d.). *Free AI Photo Enhancer: Increase Image Quality Online*. https://www.artguru.ai/